# Activity 5 Report: Database Maintenance and Monitoring

COMP 018 Database Administration | Project: Binary Sisters, Restaurant Online Ordering & Table Reservation System | Date: 6 November 2026

Run on 6 November 2026 at 22:21 (Asia/Manila), after the application click-through of Activity 3. Results come from the saved files a5_health.txt, a5_logs.txt, a5_maintenance.txt and a5_plan.txt. Items in [brackets] are for the group to fill in.

## 1. Cover and links

| Item | Value |
|---|---|
| Our group | G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph |
| Owning group | Binary Sisters |
| Repository | https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System |
| Our evidence folder | https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests |
| Tracker | https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests |

## 2. Projects monitored

| Project | Database | PostgreSQL version | Accounts used | Role served |
|---|---|---|---|---|
| Binary Sisters | grp_binarysisters_projectdb | 18.6 (x86_64 Windows) | rors_maint | Maintenance and monitoring (pg_monitor, pg_maintain) |
| | | | rors_app | Application runtime, used only to run the selected application query |

The superuser was used only to set passwords and to read the server log settings, not for the checks or maintenance.

## 3. Health and activity evidence

Commands were run as rors_maint (file a5_health.txt).

| Check | Result |
|---|---|
| PostgreSQL version and database | PostgreSQL 18.6, database grp_binarysisters_projectdb |
| Database size | 9502 kB |
| Number of tables | 8 in pg_stat_user_tables. The information_schema query returned 0 for rors_maint, because that view lists only tables the account holds privileges on, and rors_maint has none by design |
| Connections (pg_stat_database) | numbackends 2 (the monitoring session and one pgAdmin session), 1377 commits, 20 rollbacks, 0 deadlocks, 0 temp files |
| Sessions (pg_stat_activity) | Two sessions: rors_maint running the check (active) and postgres through pgAdmin 4 (idle). No rors_app session was open at that moment |

| Check | Result |
| --- | --- |
| Running queries and duration | Only the monitoring query was running. The pgAdmin session was idle (waiting on the client) |
| Dead tuples and last vacuum or analyze per table | Before this maintenance run: customer 49 dead, staff 8, payment 4, reservation 4, menu_item 2, orders 2, order_item 0, restaurant_table 0. Autovacuum had run on orders (20:53) and autoanalyze on six tables. A manual vacuum from our earlier run (21:43:13) had already cleaned orders and order_item |
| Roles still match the Activity 4 plan | Yes. Ten rors_ roles, none with superuser, CREATEDB or CREATEROLE. Connection limits 20, 2, 3, 3 and 5 on the five login roles |

Interpretation: the database is small (9502 kB) and healthy, with no deadlocks, temp files, blocked sessions or long-running queries. Row counts above the seed values (orders 62, order_item 171, payment 58, reservation 53, menu_item 51) show that data was written through the application during the Activity 3 click-through. Dead tuples were highest in customer (49 against 51 live rows). The cause was not investigated, and on a table this small the impact is negligible.

## 4. Log findings

Log source: `C:\Program Files\PostgreSQL\18\data\log` (logging_collector = on). The three newest files were searched for FATAL, ERROR, denied and failed. Server settings: log_statement = none and log_min_duration_statement = -1, so statements and slow queries are not logged.

| # | Evidence | Explanation | Impact if unresolved | Next step |
|---|---|---|---|---|
| 1 | 20:52 and 20:57: FATAL password authentication failed for user postgres | Failed postgres logins during our own setup session (a wrong password typed at a prompt). Two entries, at 20:52 and 20:57 on 6 November, both for the postgres account, before any of the tests. | None. The wrong password was rejected as intended | None. Repeated failures from an unknown source would call for blocking that address |
| 2 | 21:21:37 to 21:21:38: 17 ERROR lines, permission denied for tables orders, payment, order_item, menu_item, customer and staff, for schema public, to create database and to create role, and must be owner of table menu_item | Our own Activity 4 tests, which deliberately ran forbidden statements as rors_app, rors_report, rors_backup, rors_maint and rors_dba | None. Each denial is a privilege working as designed | None |
| 3 | 21:23 to 21:24: FATAL permission denied for database postgres (3 times) and template1 (once) | Our own Activity 4 test connecting as rors_app to other databases | None. It shows CONNECT is not granted outside the project database | None |
| 4 | 5 November 22:21 and 6 November 21:59: fourteen FATAL entries, database grpxx_canteendb does not exist | Connection attempts from a pgAdmin terminal tab on our own PC to a database of an unrelated project that does not exist on this server. Not related to Binary Sisters | None for this project | Close or correct that pgAdmin tab |

| # | Evidence | Explanation | Impact if unresolved | Next step |
|---|---|---|---|---|
| 5 | 6 November 00:03: ERROR canceling statement due to user request | A statement was cancelled by a user | None | None |

Limitations: with log_statement = none the log shows errors and failed connections but not which queries ran, and only the three newest log files were read. Two further denials at 21:43:16 (customer and payment) came from our Activity 6 access test.

## 5. Maintenance performed

Run as rors_maint, each command standalone, and VACUUM FULL was not used (file a5_maintenance.txt).

| Command | Why | Result |
|---|---|---|
| ANALYZE | Refresh planner statistics for every table after the application test activity | Completed without errors |
| VACUUM (VERBOSE, ANALYZE) public.orders | orders is the most written table. Reclaims dead rows and refreshes statistics | 2 dead row versions removed, 62 remain, 2 pages scanned. n_dead_tup 2 to 0 |
| VACUUM (VERBOSE, ANALYZE) public.order_item | Written together with orders and the largest table by rows | 0 dead row versions, 171 remain, 2 pages scanned. n_dead_tup stayed 0 |

The maintenance account ran these commands without superuser rights because it holds pg_maintain, which exists on PostgreSQL 17 and newer (the server is 18.6). Verification: pg_stat_user_tables shows last_vacuum and last_analyze of 22:22:00 for both tables and 0 dead tuples.

## 6. Query performance analysis

Query (run as rors_app, the account the application uses for the My orders page):

```
SELECT * FROM orders WHERE customer_id = 1 ORDER BY order_date DESC;
```

| Item | Result |
|---|---|
| Scan type | Seq Scan on orders under a Sort node. idx_orders_customer_date was not used |
| Estimated vs actual rows | Estimated 1, actual 0. customer_id = 1 has no orders, and 62 rows were removed by the filter |
| Execution time | 0.056 ms (planning 1.754 ms) |
| Buffers | shared hit 5, no disk reads |

Interpretation: the table has 2 pages (62 rows), so reading it whole costs less than an index lookup and PostgreSQL correctly chose a sequential scan. The plan is reasonable for this size. No new index is proposed, because the project already defines idx_orders_customer_date and each extra index adds write cost. The index becomes useful as the table grows.

## 7. Recommendations

| Priority | Recommendation |
|---|---|
| Immediate action | None. No errors beyond our own tests, no long-running queries and no blocked sessions |
| Follow-up action | Set log_min_duration_statement (for example 500 ms) and consider log_statement = ddl, so slow queries and schema changes are logged. Re-run VACUUM and ANALYZE after large data loads. Add the missing 03_set_passwords.psql so the accounts work right after setup |
| No action required | Database size, connection use (2 of 100 max_connections), role attributes, dead tuples after vacuum, and query time on the sample data |

## 8. Limitations and permissions

- rors_maint cannot read application data (confirmed in Activity 4), so table information comes from pg_stat_user_tables. information_schema.tables returned 0 tables for this account for the same reason.

- pg_monitor lets rors_maint see other users' queries in pg_stat_activity. During the check it could see the idle pgAdmin session of the postgres account in addition to its own.
- Logging detail depends on the server settings: log_statement = none and log_min_duration_statement = -1.

## 9. Feedback sent to owners

Sent on 6 November 2026: no health problems found. Recommended enabling slow-query logging, re-running VACUUM and ANALYZE after large loads, and adding the missing 03_set_passwords.psql.

## 10. AI usage declaration

- **Tool:** Claude (Anthropic).
- **Tasks:** interpreting the results of the testing.
- **Validation:** the group ran the script on its own server, and every result in this report comes from the saved output files.
- **Correction made:** the AI first expected information_schema.tables to show 8 tables for rors_maint. The output showed 0 because that account has no table privileges, so pg_stat_user_tables was used.